Assignment #4

Evaluating Expressions

Daniel Martins Cabrita de Sousa: 66128

Professor: Wellington Júnior

Assignment report for Parallel and concurrent programming
MSc in Software Engineering

December 2025

Chapter 1

Architecture

This chapter details the parallel architecture and design decisions made for the GPU-accelerated evaluation of mathematical expressions using PyTorch.

1.1 Parallelization Strategy

1.1.1 Data and Task Parallelism

Data Parallelism: All input columns and target column are transferred to GPU once at dataset load and remain resident as tensors. Vectorized operations process entire arrays simultaneously with no per sample loops on device, and each elementwise operation benefits from parallel threads across the sample dimension. **Task Parallelism:** Multiple candidate expressions are batched (up to 32 per batch via `TASK_BATCH`). Batched expressions are executed together with `torch.stack`, enabling fused kernels instead of individual per expression kernels, as PyTorch automatically fuses multiple elementwise operations into a single kernel invocation.

1.2 Kernel Design and Execution

1.2.1 Kernel Count per Batch

For each batch containing up to 32 expressions, the implementation generates approximately two elementwise kernels: one for prediction generation and one for MSE reduction. Across a complete dataset evaluation, this results in roughly 2 kernels multiplied by $\lceil n_{\text{functions}}/32 \rceil$, where the ceiling function accounts for the final potentially incomplete batch.

1.2.2 Thread and Block Selection

Thread and block dimensions are delegated entirely to the PyTorch CUDA backend, which automatically selects optimal grid and block configurations for elementwise operations based on the target GPU architecture. This eliminates the need for manual `<<<grid, block>>>` configuration and ensures that the implementation benefits from PyTorch's architecture specific optimizations.

1.2.3 Shared Memory Usage

The current implementation does not explicitly utilize shared memory, as the workload consists primarily of simple elementwise mathematical operations that lack the data reuse patterns typically required to benefit from shared memory optimizations. Instead, the backend relies on caches and register level reuse to maintain performance.

1.2.4 Control Flow and Divergence

Branch Divergence Handling

The expressions in this implementation evaluate to branch free arithmetic operations, resulting in no in kernel control flow and negligible warp divergence due to uniform execution paths across threads. Branching only occurs in the Python host code during best score tracking updates, ensuring that kernel execution remains divergence free and efficient.

Chapter 2

Implementation

This chapter describes the implementation details of the GPU accelerated evaluation of mathematical expressions using PyTorch, focusing on memory management, kernel call optimization, and the expression evaluation pipeline.

2.1 Memory Management

2.1.1 Host-to-Device Transfers

At initialization, all input columns and the target column (`y`) are transferred to device tensors once at dataset load and remain resident for the duration of all subsequent evaluations, eliminating the need for repeated transfers. During the evaluation loop, all expression computations occur on the device, with only scalar `.item()` reads performed for best MSE tracking, ensuring minimal host device communication overhead.

2.1.2 Data Residency

Expressions are compiled into torch operations that work directly on device tensors, ensuring that no data transfers occur between host and device during the evaluation loop. This design minimizes bandwidth usage and keeps memory transfer latency negligible throughout execution.

2.2 Kernel Call Optimization

2.2.1 Batching Strategy

Expressions are grouped into batches of up to 32 (configurable via `TASK_BATCH`), with each batch stacked into a single 2D tensor for vectorized evaluation. Since kernel launches carry fixed overhead costs, batching reduces the frequency of launches by processing multiple expressions per kernel invocation. By distributing this launch overhead across more computational work, throughput is substantially improved for large populations.

2.2.2 Pure Tensor Math

All computation occurs as chained tensor operations without per row Python loops on the GPU. The Python runtime handles only expression parsing and batch orchestration, delegating the actual computation to the CUDA backend.

2.3 Expression Evaluation Pipeline

The expression evaluation pipeline consists of five sequential steps:

1. **Parse:** `generate_kernel_code` replaces function names with `OPS[...]` (torch ops) and column references with `X['col']`.
2. **Compile:** `compile_kernel` uses `exec` to build a Python callable (`kernel_func`); same effect as the previous `eval()` step.
3. **Execute:** In `benchmark_parallel`, each compiled callable is applied to the current batch of tensors (`kernel(X, OPS)`), all resident on device.
4. **Reduce:** `errs = torch.mean((preds - y) ** 2, dim=1)` computes per-expression MSE vectorized on the device.
5. **Track:** Only `err.item()` (scalar) is read back to the host to update `best_err`/`best_expr`, minimizing transfers.

2.4 Genetic Programming Adaptations

Inputs and the target are transferred to the GPU once per dataset and remain resident for all evaluations. Expressions are parsed and compiled once per run into Python callables. Populations are evaluated in batches using the same `TASK_BATCH` stacking, which distributes kernel launch overhead across multiple expressions. Only scalar MSE values are returned to the host while predictions stay on device, minimizing host device transfers.

Chapter 3

Benchmarking

This chapter outlines the benchmarking methodology and results for evaluating the performance of GPU-accelerated mathematical expression evaluation using PyTorch compared to a CPU-based implementation.

3.1 Scope and Method

- Compare CPU (sequential NumPy loop) vs GPU (batched PyTorch, `TASK_BATCH=32`).
- Measure: total runtime per dataset, average time per function (total / $n_{\text{functions}}$), best MSE expression, and speedup (CPU time / GPU time).
- Datasets: generated by `generate_inputs.py` (rows, features, and function count per case).

3.2 Environment

- Hardware: Codelab GPU instance.
- Device selected by `torch.device("cuda" if torch.cuda.is_available() else "cpu")`.

3.3 Results

Table 3.1: Benchmark Results Summary

Test Case	Rows	Funcs	CPU (s)	GPU (s)	Speedup
large_heavy	100,000	2,000	5.5208	0.4816	11.46
large_moderate	50,000	1,000	1.3574	0.1545	8.79
large_standard	100,000	1,000	2.4130	0.2855	8.45
medium_balanced	10,000	500	0.2361	0.0246	9.61
medium_complex	20,000	1,000	0.6445	0.0448	14.39
small_medium	5,000	200	0.0674	0.0062	10.96
small_simple	1,000	100	0.0321	0.0031	10.33
xlarge_extreme	200,000	2,000	10.9501	0.5911	18.53